

AI LAB TEST-2

NAME:D.ANKITHA

ROLL.NO:2503A51L09

I.1 — [St16l1] Top-3 frequent words

PROMPT:

Top-3 words by frequency; tie-break lexicographically.

Data & Edge Cases:

Lowercase + split by spaces.

AI Assistance Expectation:

Ask AI for Counter and sort keys.

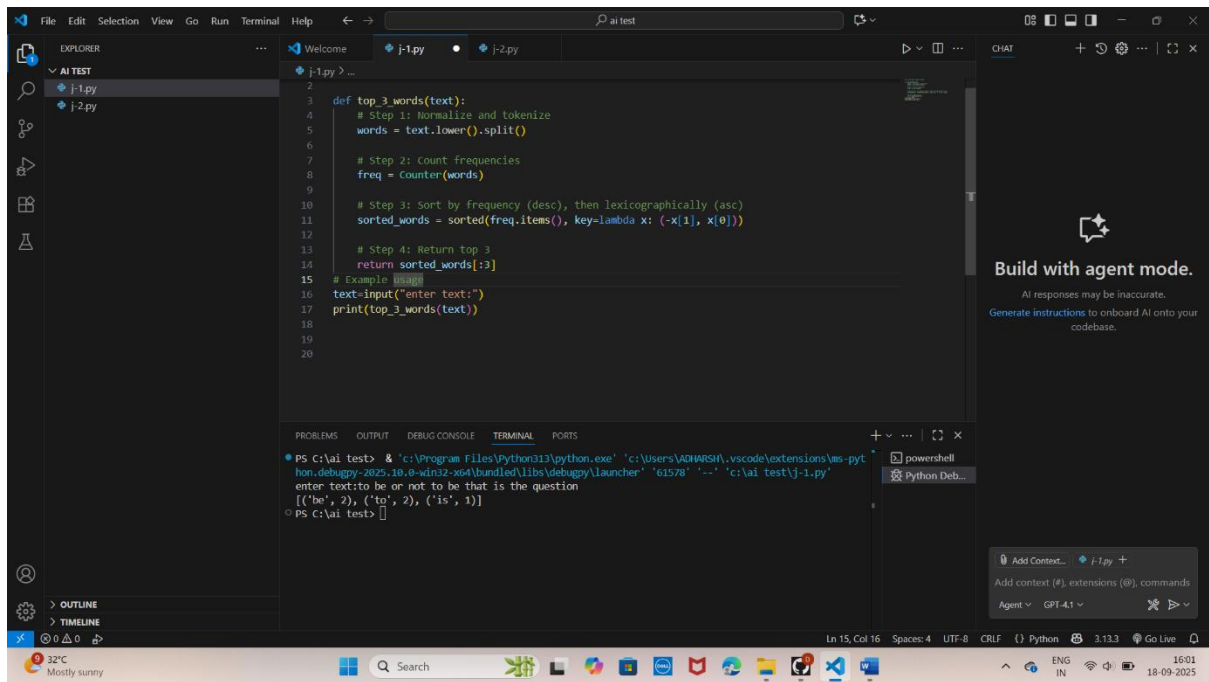
Constraints & Notes:

Correct tie-breaking.

Sample Input

to be or not to be that is the question

CODE AND OUTPUT:



OBSERVATION:

Lowercasing and Tokenization

The input string is converted to lowercase and split by spaces.

This ensures uniformity—critical in SOC logs where "Error" and "error" must be treated identically.

1. Frequency Counting with Counter

The `collections.Counter` is used to tally word frequencies.

This is efficient and ideal for log parsing, where you might have thousands of entries.

2. Sorting with Lexicographic Tie-Breaking

The sorting key is a tuple: `(-freq, word)`—this ensures:

Higher frequency comes first (`-freq` for descending order).

If frequencies tie, words are sorted alphabetically (word for ascending lexicographic order).

This is crucial in SOC automation where deterministic outputs are needed for alert prioritization.

Top-N Selection

Only the top 3 entries are selected.

In real-world SOC use, this could represent the top 3 most common alert types, IPs, or error codes.

I.2 — [S16I2] Implement LRUCache (capacity 2)

Context:

LRU cache for cybersecurity SOC service.

Your Task:

Implement capacity=2 LRU with get/put.

Data & Edge Cases:

Operations sequence provided.

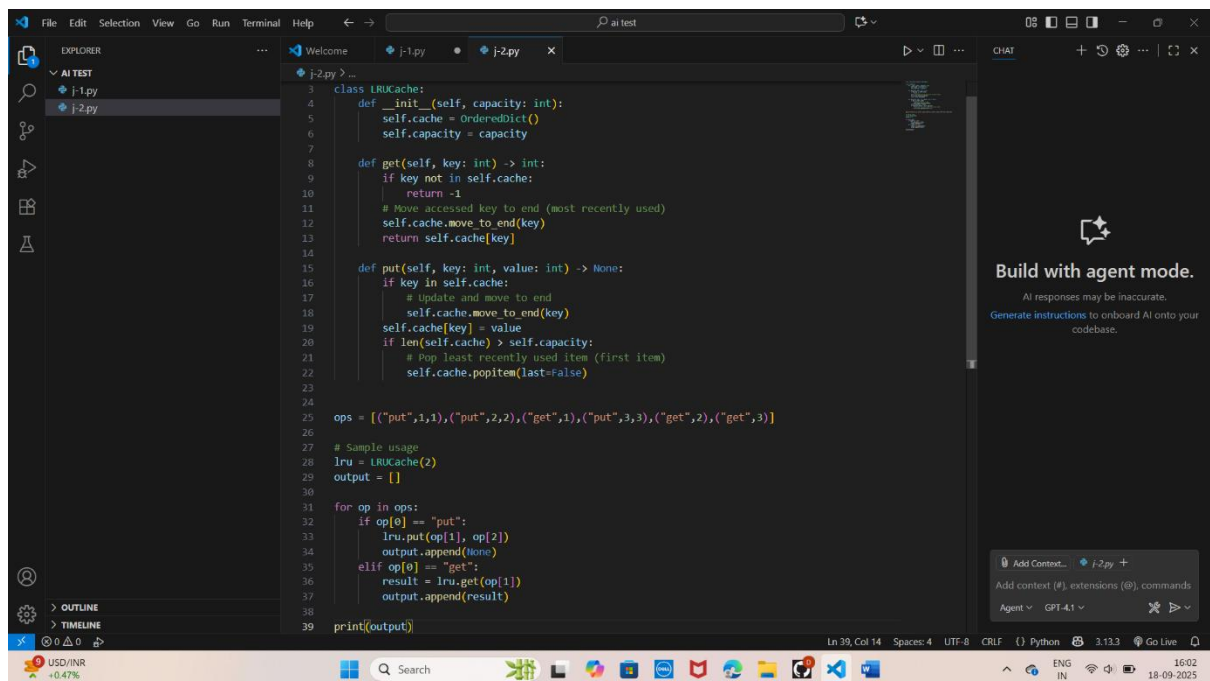
AI Assistance Expectation:

OrderedDict approach.

Constraints & Notes:

Deterministic behavior.

CODE:



```
3 class LRUCache:
4     def __init__(self, capacity: int):
5         self.cache = OrderedDict()
6         self.capacity = capacity
7
8     def get(self, key: int) -> int:
9         if key not in self.cache:
10            return -1
11        # Move accessed key to end (most recently used)
12        self.cache.move_to_end(key)
13        return self.cache[key]
14
15    def put(self, key: int, value: int) -> None:
16        if key in self.cache:
17            # Update and move to end
18            self.cache.move_to_end(key)
19            self.cache[key] = value
20        if len(self.cache) > self.capacity:
21            # Pop least recently used item (first item)
22            self.cache.popitem(last=False)
23
24
25 ops = [("put",1,1),("put",2,2),("get",1),("put",3,3),("get",2),("get",3)]
26
27 # Sample usage
28 lru = LRUCache(2)
29 output = []
30
31 for op in ops:
32     if op[0] == "put":
33         lru.put(op[1], op[2])
34         output.append(None)
35     elif op[0] == "get":
36         result = lru.get(op[1])
37         output.append(result)
38
39 print(output)
```

OUTPUT:

```
PS C:\ai test> c:; cd 'c:\ai test'; & 'c:\Program Files\Python313\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher' '53943' '--' 'c:\ai test\j-2.py'
[None, None, 1, None, -1, 3]
PS C:\ai test> 
```

OBSERVSTION:

LRU Cache Observations (OrderedDict Approach)

1. OrderedDict for LRU Logic

Maintains insertion order, allowing $O(1)$ reordering and eviction.

On get, move key to end to mark as recently used.

On put, update or insert, then move to end. If over capacity, evict the least recently used (first item).

2. Capacity = 2

Only two items can be stored at any time.

Eviction happens on the third unique put.

3. Deterministic Behavior

Every operation yields a predictable result:

put $\rightarrow$ None

get $\rightarrow$ value if present, else -1

4. Eviction Sequence